

Устройство мессенджера

Составляющие мессенджера

- Участники чата (Ноды или Узлы): N устройств с установленным приложением, состоящие в одном чате.
- Каждый участник чата имеет:
 - Пару ключей Ed25519 для подписей.
 - Пару ключей Curve25519 для E2EE шифрования частей сообщения.
 - Local Secret Key уникальный генерируемый ключ, никогда не покидающий устройство.
- Центральный Сервер: Устройство с белым IP, рассылающее трафик по нодам и проверяющее корректность запроса.

Краткое объяснение принципа работы

- Центральный сервер в данной криптосистеме выполняет роль маршрутизатора, пересылающего сообщения между нодами.
- Отправитель сообщения разделяет секрет между участниками чата на частицы L1, чтобы сервер не видел ничего.
- Каждый участник чата (узел), получив частицу L1, «запутывает» её своим ключом и снова разрезает на части L2.
- Сохраняя у себя L2 одну из частей, он рассылает все остальные частицы L2 участникам чата, которые сохраняют их у себя.
- То есть для чтения сообщения нужно не просто захватить K участников чата, а собрать K групп суб-частиц и иметь доступ к K локальным ключам.

Актуальность

- Основное отличие от современных мессенджеров заключается в хранении участниками чата (нодами) сообщений данного чата.
- Без набранного порога участников K этого чата онлайн, прочитать эти сообщения будет невозможно, потому что каждый участник хранит часть от части всего сообщения.
- Мессенджер построен для противодействия MITM, SSL-корпоративных атак и взлома конкретного участника чата для быстрого получения всех сообщений.
- Если N-K+1 участников чата удалят этот мессенджер со своих устройств, то восстановить данные чата для остальных участников будет невозможно.

Используемая математика

- **Поля Галуа $GF(2^8)$ ($[0,255]$) и примитивный полином $0x1D$**
 - Ограничивает все математические операции (сложение, умножение, деление) диапазоном $[0,255]$ для 1 байта, так как сообщение шифруется побайтово.
 - $0x1D$ - полином $x^8+x^4+x^3+x^2+1$, неприводимый многочлен, используемый в стандарте AES.
 - Он гарантирует, что при «переполнении» во время умножения байтов мы получим равномерное распределение результатов без потерь информации.
 - Любая частица сообщения имеет тот же размер, что и исходный байт.
- **Двухслойная Схема Шамира с порогом K**
 - С точки зрения математики, через K-1 точку может проходить бесконечное количество кривых, и любая точка на оси Y равновероятна.
 - Также SSS не опирается на сложность задачи, поэтому она устойчива к взлому на квантовых компьютерах.
- **Ed25519 (Цифровые подписи)**
 - Это современная схема подписи на базе эллиптических кривых Эдвардса.
 - Быстрее RSA и защищена от атак на сторонние каналы и ошибок генерации случайных чисел.
 - Подписи короткие, что критично для мессенджера, где каждое сообщение должно быть подписано для защиты от подделки со стороны сервера-ретранслятора.
- **Curve25519 / X25519 (E2EE Шифрование частиц)**
 - Для шифрования каждой частицы публичным ключом узла.
- **SHA-512 (Контрольные суммы)**
 - Перед разделением сообщения на части, клиент вычисляет хеш исходного сообщения.
 - Этот хеш рассылается всем нодам вместе с метаданными.
 - При восстановлении из суб-частиц, если одна из нод пришлет «битую» или поддельную частицу, математика Шамира выдаст неверный байт.
 - Хеш позволяет мгновенно верифицировать результат и понять, какой именно узел предоставил ложные данные.

Структура чата

- У каждого участника (узла) есть свой ключ.
- Участник (узел) знает хеши публичных ключей всех остальных узлов.
- Чат локально хранит суб-частицы на каждом клиенте в защищенном хранилище, запоминая от какого участника эти частицы пришли.

Генерация и состав «Цифрового Паспорта» участника чата

При создании нового чата, узел (участник) локально генерирует набор ключей.

- Identity Key (Ed25519): Пара для подписи.

- Encryption Key (Curve25519): Пара для шифрования частиц.
- Node ID: Вычисляется как `SHA-512(Ed25519_PublicKey)`

QR-код содержит структуру:

1. Node_ID (Идентификатор).
2. Ed25519_PubKey
3. X25519_PubKey
4. Server_Addr (IP/Домен центрального сервера).

Добавление участника в чат (через QR-код)

При первом добавлении человека в чат вы обмениваетесь публичными ключами через QR-код.

Процесс происходит при личной встрече. Это единственный способ на 100% защититься от MITM на этапе знакомства.

Алгоритм:

1. Участник А открывает приложение и показывает QR-код.
2. Участник Б сканирует QR своим приложением.
3. Смартфон Участника Б сохраняет в защищенную базу (SQLCipher или Keystore): связку: `Node_ID_A -> {Ed25519_PubKey_A, X25519_PubKey_A}`.
4. Участник Б показывает свой QR, Участник А сканирует его.

Содержание сообщения

Слой 1

На самом глубоком уровне находится L1-частица Шамира.

- Один байт (у-координата точки на параболе в поле $GF(2^8)$).

Слой 2, E2EE

Чтобы даже Центральный Сервер не видел этот байт, мы упаковываем его в асимметричный шифр.

- Curve25519 (X25519).
- Клиент берет Публичный ключ Узла Получателя и шифрует частицу.

Слой 3, Цифровая печать

- Ed25519.
- Состав блока:
 - Зашифрованный блок из Слая 2.
 - Metadata: Secret_ID (название чата), Timestamp (время), Nonce (номер блока).
- Весь этот блок подписывается Приватным ключом Клиента.

Слой 4

Этот слой видит Центральный Сервер. Он нужен для маршрутизации.

- Протокол: Protobuf (gRPC).
 - Target_Node_ID: Кому доставить.
 - Sender_ID: От кого.
 - Payload: Слой 3.
- Весь этот блок подписывается Приватным ключом Клиента.

Что происходит при рекурсии

Когда Узел №1 «разбирает» матрешку до Слая 1, он не сохраняет байт `0xAF`. Он делает новое сообщение для своих соседей:

- Берет `0xAF`.
- Применяет свой Local Secret Key: `0xAF ^ Local_Key`.
- Разрезает результат на L2 по схеме Шамира.
- Для каждой субчастицы создает новую аналогичную (Шифрует публичным ключом соседа -> Подписывает своим ключом -> Шлет через Сервер).

Почему выбрана такая вложенность:

- Отсутствие доверия к Серверу: Сервер видит только метаданные маршрута.
- Отсутствие доверия к соседям по чату: Только целевой узел может вскрыть свою порцию данных.
- Защита от подмены: Любая модификация данных посредником обнаруживается мгновенно.
- Защита от взлома ноды: Даже если частица вскрыта полностью, внутри не финальный ответ, а лишь зашифрованный фрагмент, который снова разбит на части.

Протокол аутентификация сессии (Узел <-> Сервер)

При каждом новом подключении к серверу:

- Сервер делает проверку владения.
- Используется Ed25519. Это гарантирует, что соединение (gRPC Stream) открыто именно владельцем `Node_ID`. Сервер теперь знает: «Все пакеты

для Node_ID_1 нужно слать в этот канал».

Протокол проверки отправителя (Узел <-> Узел)

Когда Участник Б получает частицу от Участника А:

1. Слой 4 содержит Sender_ID.
 2. Участник Б достает из локальной памяти Ed25519_PubKey_A.
 3. Проверяет подпись Слоя 3.
- Это гарантирует, что сервер не подкинул свое сообщение. Только Участник А мог создать валидную подпись.

Протокол адресного шифрования (Узел <-> Узел)

Когда Отправитель готовит частицы L1:

1. Он использует X25519_PubKey каждого узла, полученного через QR.
 2. Шифрует байт методом X25519 (Diffie-Hellman).
- Это создает E2EE-канал для каждого конкретного байта. Даже если сервер или другие участники перехватят этот пакет, они не смогут его расшифровать без приватного ключа целевой ноды.

Сетевая архитектура Центрального Сервера

- На центральном сервере хранится «белый список»: ID узла -> Публичный ключ (Ed25519).

Протокол gRPC и Streams

- gRPC работает поверх HTTP/2, что позволяет держать постоянные двусторонние соединения streams.
- Узлы сами иницируют соединение с сервером.

Регистрация на Центральном Сервере

- Чтобы сервер понимал, кому пересылать трафик, он должен знать публичные ключи участников.
- Но сервер не должен иметь возможности подменить ключ.

Протокол записи:

1. Узел подключается к серверу и иницирует процедуру Register.
2. **Передача ключа:** Узел отправляет свой Node_ID и Ed25519_PublicKey.
3. **Проверка владения:** Сервер присылает случайное число (Challenge). Узел подписывает его своим приватным ключом. Если подпись верна, сервер записывает этот публичный ключ в свою базу навсегда.
4. **Контроль со стороны Клиентов:** Когда Участник Б начнет общаться с Участником А через сервер, он сравнит ключ Участника А, полученный от сервера, с тем ключом, который он лично получил через QR.
 - Если сервер попытается подsunуть «поддельный ключ» — клиент Б мгновенно обнаружит подделку.

Протокол авторизации на Центральном Сервере

- Сервер шлет случайное число (nonce), которое узел подписывает.
- Сервер проверяет подпись с помощью публичного ключа из списка.
- Если подпись верна, сервер открывает gRPC Stream и связывает его с ID узла.

Протокол работы Сервера

- Перед тем как отправить частицу в сеть, клиент шифрует её публичным ключом соответствующей Ноды.
- Если принимающий узел онлайн, у него открыт стрим, сервер просто перекладывает пакет в его канал.
- Если принимающий узел офлайн, сервер может придержать сообщение у себя на пару часов, ожидая от него сигнала готовности.

Протокол работы отправки сообщения (записи его на ноды)

1. **Клиент (Отправитель):**
 - Берет сообщение P.
 - Делит каждый байт на N частиц первого уровня (L1_1, L1_2, ... L1_n) с порогом K.
 - Для каждой ноды i зашифровывает адресованную ему частицу L1_i его (ноды) публичным ключом.
 - Подписывает весь пакет своим приватным ключом.
 - Шлет пакеты на Центральный Сервер, который перебрасывает их по нужным узлам.
2. **Каждая нода, состоящая в чате, получившая свою частицу сообщения:**
 - Проверяет подпись отправителя.
 - Расшифровывает частицу L1_i своим приватным ключом.
 - Заново шифрует эту частицу: Делает $L1_i \wedge LocalSecretKey_i$.
 - Делит частицу на суб-частицы (L2) и рассылает их соседям (другим нодам в чате) на хранение.
 - Итог: На каждой ноде остается 1 суб-частица от всего сообщения.

Протокол работы получения сообщения (чтение его с нод)

1. **Клиент (Получатель):**
 - Авторизуется на Центральном Сервере через цифровую подпись.

- Запрашивает у нод их частицы первого уровня.
2. **Ноды, совершающие процесс восстановления сообщения:**
- Узел i опрашивает соседей, собирает K суб-частиц ($L2$), зашифрованных его публичным ключом и подписанных соответствующим соседом.
 - Расшифровывает своим приватным ключом суб-частицы и восстанавливает зашифрованную частицу из набора суб-частиц соседей.
 - Снимает свой слой шифрования через $XOR LocalSecretKey_i$.
 - Шифрует частицу публичным ключом $L1_i$ клиента.
 - Отдает частицу $L1_i$ клиенту.
3. **Клиент (Получатель):**
- Собирает K частиц $L1$ от разных узлов.
 - Расшифровывает их своим приватным ключом и восстанавливает сообщение.

Протокол удаления сообщения

- Отправитель посылает через Центральный Сервер команду DELETE всем участникам чата с ID сообщения, подписанную своим ключом.
- Каждая нода физически затирает суб-частицу $L2$ данного сообщения в своей памяти/базе.

Анализ трафика

- Ноды могут слать пустые зашифрованные пакеты через случайные промежутки времени, чтобы скрыть реальную активность.

Возможные атаки

Что атакуется?	Что получают?	Как мессенджер противодействует этому?
Центральный сервер	Зашифрованные потоки байт.	У злоумышленника нет приватных ключей узлов и нет Master-пароля.
Хозяин одного узла	Зашифрованную базу - случайные суб-частицы ($L2$) и локальный ключ.	Злоумышленнику нужно взломать ещё $K-1$ узлов и украсть их локальные ключи. Также без авторизации на Центральном Сервере и получения суб-частиц от соседей он не соберет ключ.

Почему это безопасно?

1. Информационная избыточность: Чат живет, пока живы любые K узлов чата.
2. Полагаемся на то, что "параболу" невозможно восстановить по одной/двум точкам.